

DEMANDTEST: Demand-Driven Static Discovery of Project-Specific Knowledge for Efficient Intention-Aligned Unit Test Generation

CS 527 Project Proposal (Group 5)

Yunqi Li (NetID: yunqili4), Jinjie Guo (NetID: jinjie2)
University of Illinois Urbana-Champaign
{yunqili4,jinjie2}@illinois.edu

Abstract

Writing a unit test for a real project requires project-specific knowledge, and repository agents find it by reading files one at a time, at a high cost. DEMANDTEST instead computes what a test needs from the focal method’s types and the test intention, fetches only that knowledge from a static project index, stops once every need is covered, and then calls a local model once.

Keywords

Unit Test Generation, Large Language Model, Static Analysis

1 Target Problem and Motivation

Task. Given a focal method in a large Java codebase and a natural-language *validation intention*, generate a unit test that compiles, runs, and checks that intention [4]. This is Challenge 8 of the ICSE 2027 Industry Challenge Track [1].

Why it is hard. The test needs knowledge that is spread across many files: how to build valid inputs, which fixtures and mocks to use, and what to assert.

Why current tools fall short. Agents such as OpenHands [5] read files one by one, so tokens, time, and files inspected all grow with the search. Context-injection tools such as KTester [3] and CATGen [2] add fixed patterns of context, with no rule for when enough has been collected.

Goal. Comparable quality with at least 50% fewer tokens and 20% less time than an agent, on a private model.

2 Plan for Approach Design

Key idea. What a test needs is largely determined by types and by the intention, so we compute it statically, *before* any model call:

- S0 Index once.** JavaParser records constructors, factories, builders, and the fixtures of existing tests.
- S1 List the needs.** A typed *demand set*: receiver, valid inputs with their preconditions, setup, and what to assert. Needs that cannot be resolved are flagged as gaps.
- S2 Resolve, then stop.** Map each need to its cheapest construction recipe and expand only along unresolved needs. Stop when the sufficiency rule Σ holds (every need has a recipe); otherwise fall back to the two closest existing tests [6].
- S3 Generate.** One compact prompt, one call to a local open-weight model.
- S4 Check.** Static fixes, then compile and run.
- S5 Repair.** At most one repair call.

Research question. Do typed needs pick *which* context to fetch and *when to stop* better than similarity ranking?

3 Plan for Evaluation Design

- **Subjects.** The 13 Java projects of the IntentionTest benchmark [4], plus 30 new intentions; the held-out test is hidden from all systems.
- **Metrics.** *Aligned success rate*: the test compiles, passes, runs the focal method, and a human-validated judge confirms it checks the intention. Cost: tokens, time, and files inspected.
- **Baselines.** Main: *vanilla* OpenHands (default agent, prompts, tools, and iteration limit; same model, only our task prompt). Also: OpenHands capped at 10, 30, and 60 iterations; a simplified IntentionTest; a static-context baseline.
- **Ablations.** Vary only context selection, or only the stopping rule.
- **Pilot.** 150 tasks on three projects decide whether to proceed.

4 Expected Outcome

- **Main result.** Aligned success within 5 points of vanilla OpenHands at less than 50% of its tokens and 80% of its time, shown as a plot of success versus tokens, with the capped runs as extra points.
- **Deliverables.** The generator, a reusable cost-accounting harness, the reproduced benchmark, and a poster.
- **Risks.** Σ holds but the test still fails; preconditions that static analysis cannot capture; projects that do not build.

References

- [1] 2026. Efficient Project-Specific Test Generation with Validation Intentions. ICSE 2027 Industry Challenge, accepted challenge #8. <https://conf.researchr.org/track/icse-2027/icse-2027-industry-challenge>.
- [2] Junjie Chen, Ziqi Wang, Lin Yang, Chen Yang, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Dong Wang. 2026. Context Matters: Improving the Practical Reliability of LLM-Based Unit Test Generation. *arXiv preprint arXiv:2607.19682* (2026).
- [3] Anji Li, Mingwei Liu, Zhenxi Chen, Zheng Pei, Zike Li, Dekun Dai, Yanlin Wang, and Zibin Zheng. 2026. Knowledge Matters: Injecting Project and Testing Knowledge into LLM-based Unit Test Generation. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*.
- [4] Binhang Qi, Yun Lin, Xinyi Weng, Yuhuan Huang, Chenyan Liu, Hailong Sun, Zhi Jin, and Jin Song Dong. 2025. Intention-Driven Generation of Project-Specific Test Cases. *arXiv preprint arXiv:2507.20619* (2025).
- [5] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR)*.
- [6] Xiaoxuan Zhou, Yiling Lou, Jinhao Dong, and Dan Hao. 2026. TestTailor: Generating High-Coverage Tests via Path-Proximal Tests with LLMs. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 277–298. doi:10.1145/3797140